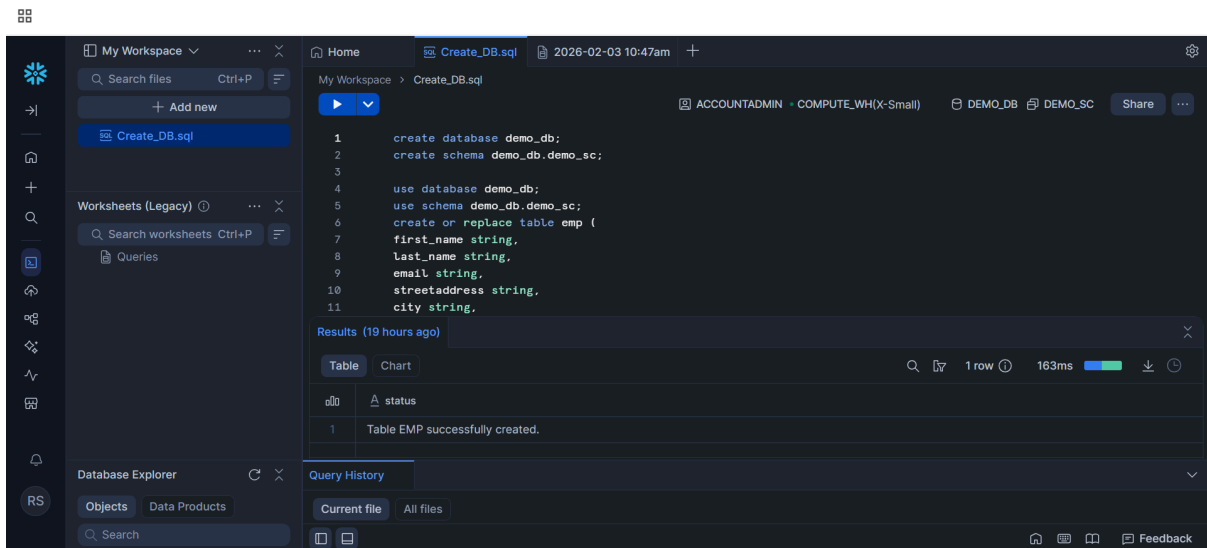


03/02/2026

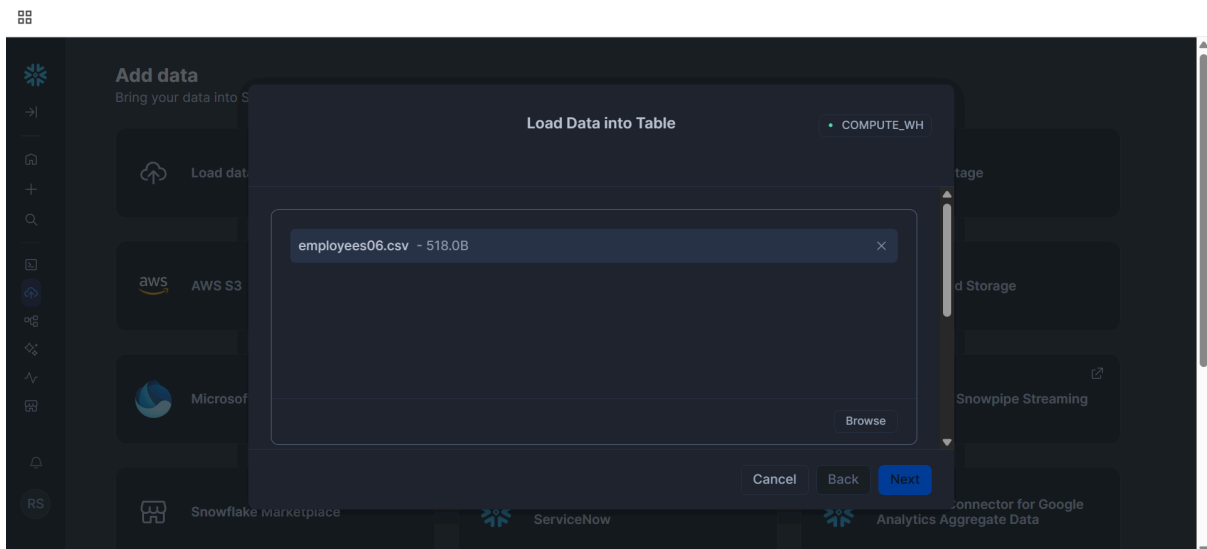
Snowflake Screenshots

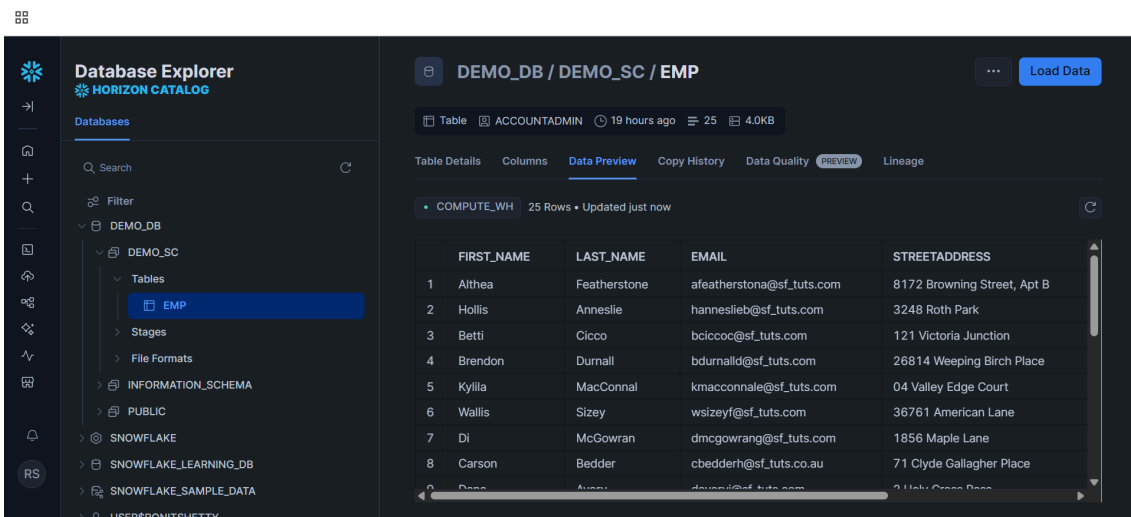
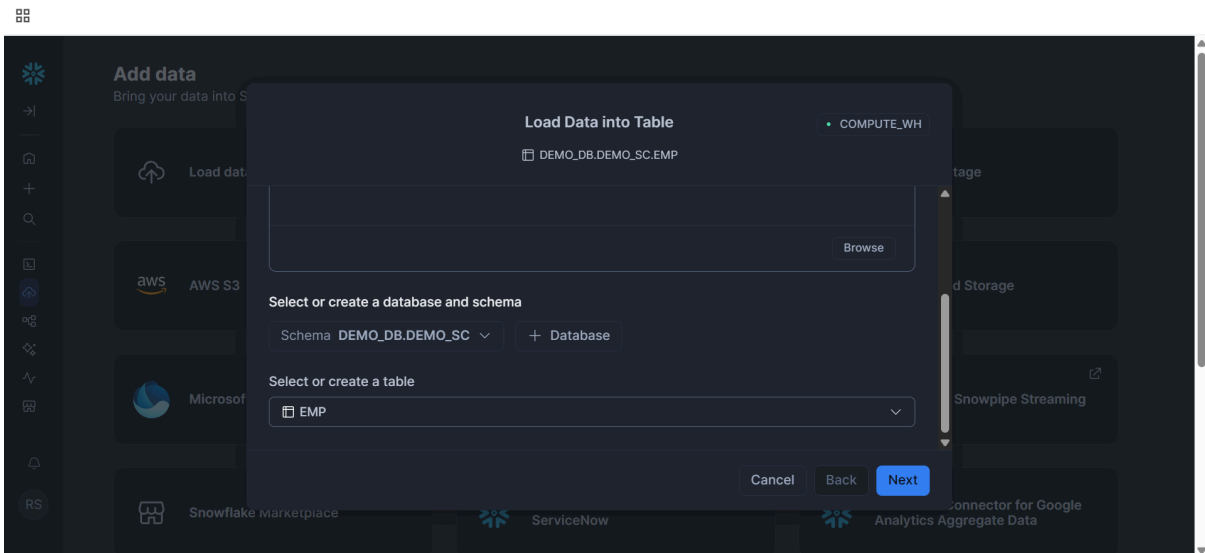
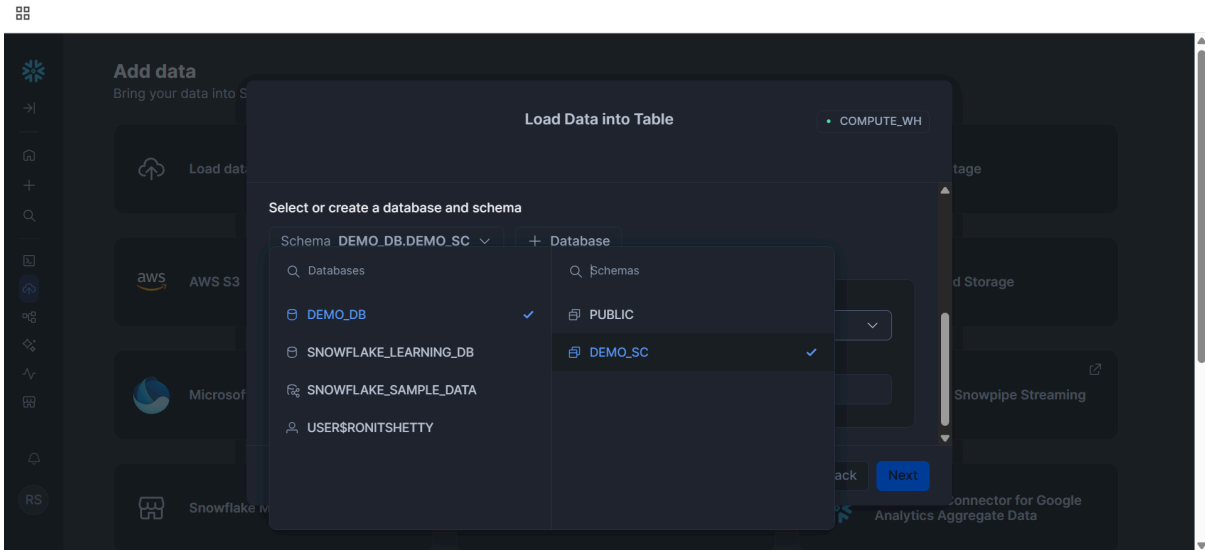
Name: Ronit Shetty

Creating database, schema and table



Loading data through web UI





Running queries

Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$398 credits left

Trial ends in 29 days

Upgrade

RS Ronit Shetty

ACCOUNTADMIN

My Workspace

Search files Ctrl+P

+ Add new

Create_DB.sql

Worksheets (Legacy)

Search worksheets Ctrl+P

Queries

Database Explorer

Objects Data Products

Search

Home

Create_DB.sql

2026-02-03 10:47am

Queries

1 SELECT * FROM "DEMO_DB"."DEMO_SC"."EMP" LIMIT 10;

2

3 SELECT COUNT(*) AS total_employees FROM "DEMO_DB"."DEMO_SC"."EMP";

4

5 | Ctrl+I to generate

Results (1 minute ago)

Table Chart

1 row 46ms

	TOTAL_EMPLOYEES
1	25

Query History

Current file All files

Feedback

Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$398 credits left

Trial ends in 29 days

Upgrade

RS Ronit Shetty

ACCOUNTADMIN

My Workspace

Search files Ctrl+P

+ Add new

Create_DB.sql

Sample_Queries.sql

Worksheets (Legacy)

Search worksheets Ctrl+P

Queries

Database Explorer

Objects Data Products

Search

Home

Create_DB.sql

2026-02-03 10:47am

Queries

Sample_Queries.sql

1 SELECT

2 MONTHNAME(start_date) as hiring_month,

3 COUNT(*) as hires

4 FROM "DEMO_DB"."DEMO_SC"."EMP"

5 WHERE YEAR(start_date) = 2017

6 GROUP BY hiring_month

7 ORDER BY hires DESC;

8

9 SELECT

10 CONCAT(first_name, ' ', last_name) as full_name,

11 email

Results (3 minutes ago)

Table Chart

11 rows 115ms

HIRING_MONTH		HIRES
Apr	9.1%	
Aug	9.1%	

Query History

Current file All files

Feedback

Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$398 credits left

Trial ends in 29 days

Upgrade

RS Ronit Shetty

ACCOUNTADMIN

My Workspace

Search files Ctrl+P

+ Add new

Create_DB.sql

Sample_Queries.sql

Worksheets (Legacy)

Search worksheets Ctrl+P

Queries

Database Explorer

Objects Data Products

Search

Home

Create_DB.sql

2026-02-03 10:47am

Queries

Sample_Queries.sql

10 SELECT

11 CONCAT(first_name, ' ', last_name) as full_name,

12 email,

13 city

14 FROM "DEMO_DB"."DEMO_SC"."EMP"

ORDER BY last_name ASC;

Results (6 minutes ago)

Table Chart

25 rows 72ms

FULL_NAME	EMAIL	CITY
Althea Featherstone	adavidovtsk@sf_tuts.com	Aleysk
Arlene Davidovits	afeatherstona@sf_tuts.com	Banjar Bengkelgede
+23 more	+23 more	+23 more
Hollis Anneslie	hanneslieb@sf_tuts.com	Aleysk
Dana Avory	davoryi@sf_tuts.com	Wenlin
Granger Bassford	gbassfordo@sf_tuts.co.uk	Karditsa

Query History

Feedback



Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$398 credits left

Trial ends in 29 days

Upgrade

RS Ronit Shetty

ACCOUNTADMIN

My Workspace

Search files Ctrl+P

+ Add new

Basic_Queries.sql

Create_DB.sql

Sample_Queries.sql

Worksheets (Legacy)

Search worksheets Ctrl+P

Queries

Database Explorer

Objects

Data Products

Search

Create_DB.sql

2026-02-03 10:47am

Queries

Sample_Queries.sql

Basic_Queries.sql

My Workspace > Basic_Queries.sql

1 -- 1. Simple Filtering (WHERE)

2 -- Get all employees who live in 'Miami' or have 'Miami' in their address

3 SELECT * FROM "DEMO_DB"."DEMO_SC"."EMP"

4 WHERE city = 'Miami'

5 OR streetaddress LIKE '%Miami%';

6

7 -- 2. Sorting Data (ORDER BY)

8 -- List employees alphabetically by their Last Name

9 SELECT first_name, last_name, start_date

Results (just now)

Table Chart

1 row 2.0s

	FIRST_NAME	LAST_NAME	EMAIL	STREETADDRESS	CITY
1	Cesar	Hovie	chovie9@sf_tuts.com	5 7th Pass	Miami

Query History



Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$398 credits left

Trial ends in 29 days

Upgrade

RS Ronit Shetty

ACCOUNTADMIN

My Workspace

Search files Ctrl+P

+ Add new

Basic_Queries.sql

Create_DB.sql

Sample_Queries.sql

Worksheets (Legacy)

Search worksheets Ctrl+P

Queries

Database Explorer

Objects

Data Products

Search

Create_DB.sql

2026-02-03 10:47am

Queries

Sample_Queries.sql

Basic_Queries.sql

My Workspace > Basic_Queries.sql

4 WHERE city = 'Miami'

5 OR streetaddress LIKE '%Miami%';

6

7 -- 2. Sorting Data (ORDER BY)

8 -- List employees alphabetically by their Last Name

9 SELECT first_name, last_name, start_date

10 FROM "DEMO_DB"."DEMO_SC"."EMP"

11 ORDER BY last_name ASC;

12

13 -- 3. Aggregation (GROUP BY)

Results (just now)

Table Chart

25 rows 883ms

	FIRST_NAME	LAST_NAME	START_DATE
	Althea	Anneslie	
	Arlene	Avory	
	+23 more	+23 more	
1	Hollis	Anneslie	2017-11-16
2	Dana	Avory	2017-05-11

Query History



Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$398 credits left

Trial ends in 29 days

Upgrade

RS Ronit Shetty

ACCOUNTADMIN

My Workspace

Search files Ctrl+P

+ Add new

Basic_Queries.sql

Create_DB.sql

Sample_Queries.sql

Worksheets (Legacy)

Search worksheets Ctrl+P

Queries

Database Explorer

Objects

Data Products

Search

Create_DB.sql

2026-02-03 10:47am

Queries

Sample_Queries.sql

Basic_Queries.sql

My Workspace > Basic_Queries.sql

26

27 -- 5. Date Range Filtering

28 -- Find everyone hired between Jan 1, 2017 and June 1, 2017

29 SELECT first_name, last_name, start_date

30 FROM "DEMO_DB"."DEMO_SC"."EMP"

31 WHERE start_date BETWEEN '2017-01-01' AND '2017-06-01'

32 ORDER BY start_date;

Results (just now)

Table Chart

7 rows 82ms

	FIRST_NAME	LAST_NAME	START_DATE
1	Violette	Shermore	2017-01-19
2	Carson	Bedder	2017-03-29
3	Ivett	Casemore	2017-03-29
4	Nvssa	Doraan	2017-04-13

Query History

The screenshot displays the Snowflake web interface. On the left, a sidebar shows navigation options like 'Work with data', 'Projects', 'Ingestion', 'Transformation', 'AI & ML', and 'Monitoring'. Below this, a credit balance of '\$398 credits left' is shown. The main workspace is divided into three panes. The top pane shows a list of files: 'Basic_Queries.sql', 'Create_DB.sql', and 'Sample_Queries.sql'. The middle pane shows a list of worksheets and queries. The bottom pane displays a SQL query in a workspace titled 'My Workspace > Temporary_Table_Queries.sql'. The query is as follows:

```

11 SELECT first_name, last_name, emp_date FROM emp_employee;
12
13 -- Use conditional logic to label employees as 'Veteran' or 'Newcomer' based on days
14 -- employed
15 SELECT first_name, days_employed, CASE WHEN days_employed > 3000 THEN 'Veteran' ELSE
16 'Newcomer' END as status FROM emp_analysis_temp;
17
18 -- Calculate the average employment duration (in days) for each city
19 SELECT city, AVG(days_employed) as avg_tenure_days FROM emp_analysis_temp GROUP BY city
  
```

Below the query, the 'Results (just now)' section shows a table with 25 rows and 67ms execution time. The table has columns: FIRST_NAME, #, DAYS_EMPLOYED, and STATUS. A bar chart is overlaid on the DAYS_EMPLOYED column, showing the distribution of employment durations. The results are as follows:

FIRST_NAME	#	DAYS_EMPLOYED	STATUS
Althea	4.0%	Bar chart showing distribution of days_employed	Veteran
Arlene	4.0%		Newcomer
+23 more			
1 Althea	3127		Veteran
2 Hollis	3000		Newcomer

At the bottom, a 'Database Explorer' pane shows the 'Objects' tab with 'Data Products' and a search bar.

snowflake

Work with data

- Projects
- Ingestion
- Transformation
- AI & ML
- Monitoring

\$398 credits left
Trial ends in 29 days
[Upgrade](#)

RS Ronit Shetty
ACCOUNTADMIN

My Workspace

- Search files Ctrl+P
- + Add new
- Basic_Queries.sql
- Create_DB.sql
- Sample_Queries.sql

Worksheets (Legacy)

- Search worksheets Ctrl+P
- Queries

Database Explorer

- Objects
- Data Products

02-03 10:47am Queries

Sample_Queries.sql Basic_Queries.sql Temporary_Table_Queries.sql

My Workspace > Temporary_Table_Queries.sql

```
20 SELECT city, COUNT(*) as start_count FROM emp_analysis_temp GROUP BY city HAVING
21 COUNT(*) > 1;
22
23 -- Filter specifically for employees living in apartments or suites using wildcard
24 -- matching
25 SELECT * FROM emp_analysis_temp WHERE streetaddress LIKE '%Apt%' OR streetaddress LIKE
26 '%Suite%';
27
28 -- Find the employee with the longest last name using string length calculation
29 SELECT first_name, last_name, LENGTH(last_name) as name_length FROM emp_analysis_temp
30 ORDER BY name_length DESC LIMIT 1;
```

Results (just now)

Table	Chart	1 row	67ms	Download	Refresh
#	FIRST_NAME	LAST_NAME	EMAIL	CITY	STREETADDRESS
1	Althea	Featherstone	afeatherston@sf_tuts	Calatrava	8172 Browning Street,

Query History

Feedback

snowflake

Work with data

- Projects
- Ingestion
- Transformation
- AI & ML
- Monitoring

\$398 credits left
Trial ends in 29 days
[Upgrade](#)

RS Ronit Shetty
ACCOUNTADMIN

My Workspace

- Search files Ctrl+P
- + Add new
- Basic_Queries.sql
- Create_DB.sql
- Sample_Queries.sql

Worksheets (Legacy)

- Search worksheets Ctrl+P
- Queries

Database Explorer

- Objects
- Data Products

02-03 10:47am Queries

Sample_Queries.sql Basic_Queries.sql Temporary_Table_Queries.sql

My Workspace > Temporary_Table_Queries.sql

```
24
25 -- Find the employee with the longest last name using string length calculation
26 SELECT first_name, last_name, LENGTH(last_name) as name_length FROM emp_analysis_temp
27 ORDER BY name_length DESC LIMIT 1;
28
29 -- Aggregate hiring trends to see how many people joined in each year
30 SELECT YEAR(start_date) as hire_year, COUNT(*) as total_hires FROM emp_analysis_temp
31 GROUP BY hire_year;
```

Results (just now)

Table	Chart	1 row	88ms	Download	Refresh
#	FIRST_NAME	LAST_NAME	#	NAME_LENGTH	
1	Althea	Featherstone		12	

Query History

Feedback

snowflake

Work with data

- Projects
- Ingestion
- Transformation
- AI & ML
- Monitoring

\$398 credits left
Trial ends in 29 days
[Upgrade](#)

RS Ronit Shetty
ACCOUNTADMIN

My Workspace

- Search files Ctrl+P
- + Add new
- Basic_Queries.sql
- Create_DB.sql
- Sample_Queries.sql

Worksheets (Legacy)

- Search worksheets Ctrl+P
- Queries

Database Explorer

- Objects
- Data Products

02-03 10:47am Queries

Sample_Queries.sql Basic_Queries.sql Temporary_Table_Queries.sql

My Workspace > Temporary_Table_Queries.sql

```
27 ORDER BY name_length DESC LIMIT 1;
28
29 -- Aggregate hiring trends to see how many people joined in each year
30 SELECT YEAR(start_date) as hire_year, COUNT(*) as total_hires FROM emp_analysis_temp
31 GROUP BY hire_year;
32
33 -- Filter for employees living in cities that start with the letters S or M
34 SELECT * FROM emp_analysis_temp WHERE SUBSTR(city, 1, 1) IN ('S', 'M');
```

Results (just now)

Table	Chart	2 rows	145ms	Download	Refresh
#	HIRE_YEAR	#	TOTAL_HIRES		
1	2017		21		
2	2016		4		

Query History

Feedback

The screenshot displays the Snowflake SQL Editor interface. On the left, there's a sidebar with navigation options like 'Work with data', 'Projects', 'Ingestion', 'Transformation', 'AI & ML', and 'Monitoring'. A credit balance of '\$398 credits left' is shown. The main workspace is titled 'My Workspace' and contains a file explorer with 'Basic_Queries.sql', 'Create_DB.sql', and 'Sample_Queries.sql'. The active query file is 'Temporary_Table_Queries.sql', which contains the following SQL code:

```
-- Aggregate hiring trends to see how many people joined in each year
SELECT YEAR(start_date) as hire_year, COUNT(*) as total_hires FROM emp_analysis_temp
GROUP BY hire_year;

-- Filter for employees living in cities that start with the letters S or M
SELECT * FROM emp_analysis_temp WHERE SUBSTR(city, 1, 1) IN ('S', 'M');
```

The query results are displayed in a table with 9 rows and 7 columns: FIRST_NAME, LAST_NAME, EMAIL, CITY, STREETADDRESS, and STAFF. The results show data for employees like Betti Cicco, Brendon Durnall, Catherin Devereu, and Grazia Glaserman.

Query caching enabled (default) - Result reuse

The screenshot shows the Snowflake Query Profile page for a specific query. The query ID is '01c229a9-0001-788a-0000-000d8baec619'. The page is divided into three tabs: 'Query Details', 'Query Profile' (selected), and 'Query Telemetry'. The 'Query Profile' tab shows a 'Query insights' section with 'None detected' and a 'Profile overview(Finished)' section with a progress bar. A 'QUERY RESULT REUSE' section at the bottom shows a progress bar for the query ID '01c229a9-0001-788a-0000-000d8bae...'. The page also includes a sidebar with navigation options and a user profile for 'Ronit Shetty'.

Connecting to azure:

The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the Microsoft Azure logo, a search bar, and the user profile 'Ronit Shetty@ext.saint-...'. The main content area displays the 'container1' properties page. On the left, there is a sidebar with navigation options: Overview, Diagnose and solve problems, Access Control (IAM), Settings, Shared access tokens, Access policy, Properties (selected), and Metadata. The main panel shows the following details for 'container1':

- NAME:** container1
- URL:** https://ronitshettystorage.blob.core.windows.net/container1
- LAST MODIFIED:** 2/3/2026, 1:16:27 PM
- ETAG:** 0x8DE62F856885B2
- LEASE STATUS:** Unlocked

A 'Copy' button is visible next to the URL field.

The screenshot shows the Snowflake SQL interface. The top navigation bar includes the Snowflake logo, a search bar, and the user profile 'Ronit Shetty ACCOUNTADMIN'. The main content area displays a SQL query in the 'load_to_azure.sql' file:

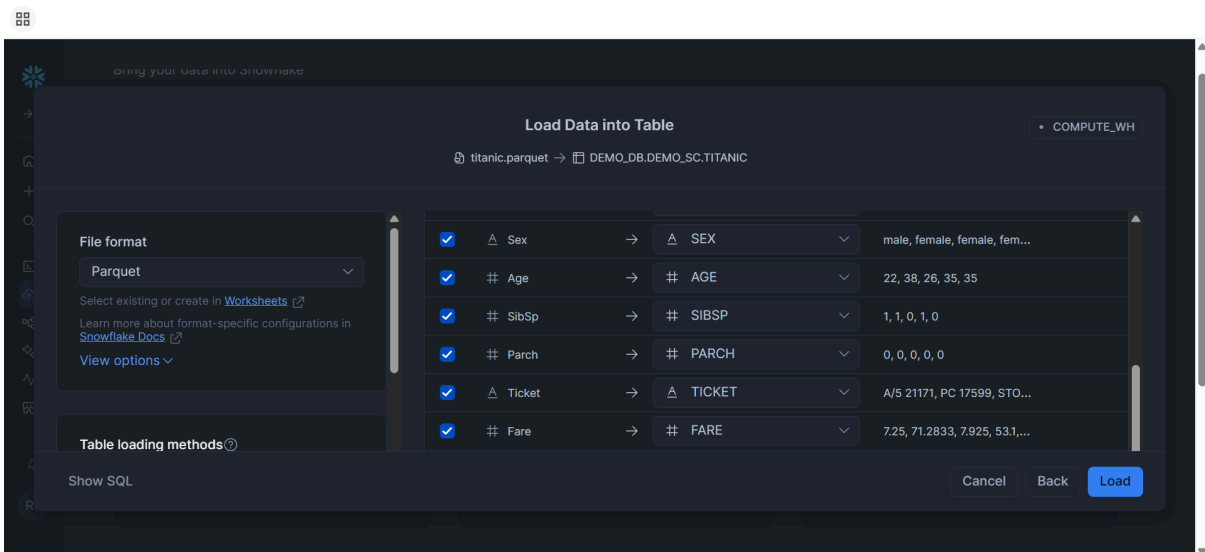
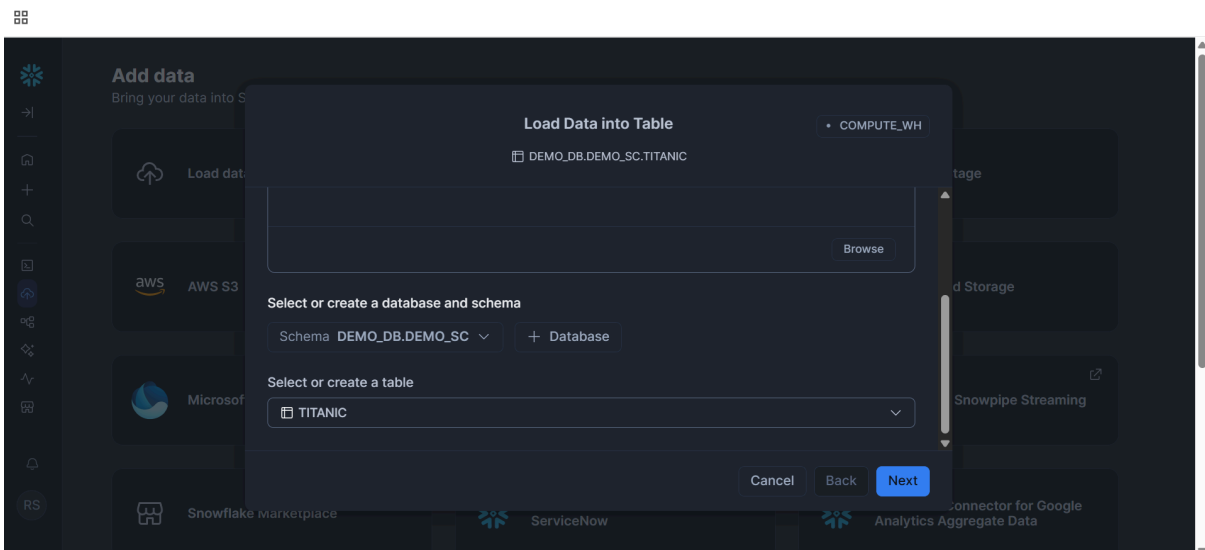
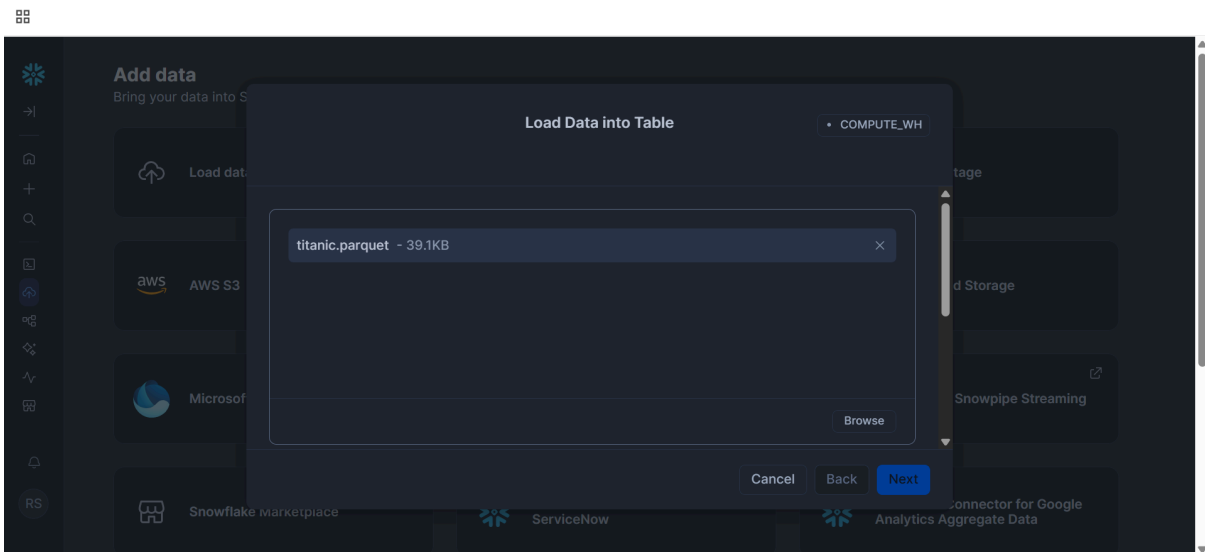
```
1 create or replace storage integration azure_int
2 type = external_stage
3 storage_provider = azure
4 enabled = true
5 azure_tenant_id = 'e339bd4b-2e3b-4035-a452-2112d502f2ff'
6 storage_allowed_locations =
7   ('azure://ronitshettystorage.blob.core.windows.net/container1');
8 desc integration azure_int;
```

The query results show a single row with the status 'Integration AZURE_INT successfully created.' The bottom panel displays the 'Query History' section.

Error with azure (snowsql is restricted too)

The screenshot shows a Saint-Gobain login page. The top navigation bar includes the Saint-Gobain logo and the user profile 'ronit.shetty@ext.saint-gobain.com'. The main content area displays a 'Need admin approval' message. The message states that the user 'otuesrsnowflakepacint' is unauthorized and needs permission to access resources in the organization. It provides a link to 'Have an admin account? Sign in with that account' and a link to 'Return to the application without granting consent'.

Titanic data loading





Database Explorer
HORIZON CATALOG

Databases

Search

Filter

DEMO_DB

DEMO_SC

Tables

EMP

TITANIC

Stages

File Formats

INFORMATION_SCHEMA

PUBLIC

SNOWFLAKE

SNOWFLAKE_LEARNING_DB

SNOWFLAKE_SAMPLE_DATA

DEMO_DB / DEMO_SC / TITANIC

Table ACCOUNTADMIN 6 minutes ago 0 0.0B

Table Details Columns Data Preview Copy History Data Quality (PREVIEW) Lineage

COMPUTE_WH 100 Rows • Updated just now

	PASSENGERID	SURVIVED	PCLASS	NAME
1	1	0	3	Braund, Mr. Owen Harris
2	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)
3	3	1	3	Heikkinen, Miss. Laina
4	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)
5	5	0	3	Allen, Mr. William Henry
6	6	0	3	Moran, Mr. James
7	7	0	1	McCarthy, Mr. Timothy J
8	8	0	3	Palsson, Master. Gosta Leonard



Database Explorer
HORIZON CATALOG

Databases

Search

Filter

DEMO_DB

DEMO_SC

Tables

EMP

TITANIC

Stages

File Formats

INFORMATION_SCHEMA

PUBLIC

SNOWFLAKE

SNOWFLAKE_LEARNING_DB

DEMO_DB / DEMO_SC / TITANIC

Table ACCOUNTADMIN 7 minutes ago 0 0.0B

Table Details Columns Data Preview Copy History Data Quality (PREVIEW) Lineage

Last 7 days Status All Pipe All 1 Copy Search COMPUTE_WH

Loaded (1)

1

0 Jan 28 Jan 29 Jan 30 Jan 31 Feb 1 Feb 2 Feb 3

FILE NAME	LOADED	SIZE	ROWS	STATUS	SOUR...
2026-02-03T11:10:46.952Z/titanic.par...	1m ago	39.1KB	891	Loaded	

Running queries



snowflake

My Workspace

Search files Ctrl+P

+ Add new

Basic_Queries.sql

Create_DB.sql

load_to_azure.sql

Monthwise_Hires.sql

Temporary_Table_Queries.sql

Titanic_Data.sql

Verifying_Data_Load.sql

Work with data

Projects

Ingestion

Transformation

AI & ML

Monitoring

\$395 credits left

Trial ends in 29 days

Upgrade

Ronit Shetty

ACCOUNTADMIN

My Workspace

Basic_Queries.sql

Temporary_Table_Queries.sql

Verifying_Data_Load.sql

load_to_azure.sql

Titanic_Data.sql

My Workspace

Titanic_Data.sql

24

25

26

27

SELECT * FROM titanic LIMIT 10;

Results (just now)

Table Chart

10 rows 66ms

#	PASSENGERID	#	SURVIVED	#	PCLASS	NAME	SEX	#	AGE	#	SURVIVED
1	1	0	1	3	Allen, Mr. William Henry	male	22	0			
2	2	1	1	1	Braund, Mr. Owen Harris	male	22	0			
3	3	1	3	1	Cumings, Mrs. John Bradley	female	38				
4	4	1	3	1	Heikkinen, Miss. Laina	female	26				

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries.sql
- Titanic_Queries.sql**
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

- DEMO_SC
 - Tables 2
 - Stages 1

My Workspace > **Titanic_Queries.sql**

1 `SELECT COUNT(*) FROM demo_db.demo_sc.titanic;`

Results (just now)

Table	Chart	1 row	29ms
#		COUNT(*)	
1		891	

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries.sql
- Titanic_Queries.sql**
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

- DEMO_SC
 - Tables 2
 - Stages 1

My Workspace > **Titanic_Queries.sql**

```
4 SELECT COUNT(*) FROM demo_db.demo_sc.titanic;
5
6 -- 2. Survival Count: How many survived (1) vs Died (0)?
7 SELECT Survived, COUNT(*) as passenger_count
8 FROM titanic
9 GROUP BY Survived;
10
11 -- 3. The "Women and Children First" Check
12 -- Calculate survival rate by Gender
13
```

Results (just now)

Table	Chart	2 rows	120ms
#		SURVIVED	# PASSENGER_COUNT
1		1	342
2		0	549

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries.sql
- Titanic_Queries.sql**
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

- DEMO_SC
 - Tables 2
 - Stages 1

My Workspace > **Titanic_Queries.sql**

```
12 -- Calculate survival rate by Gender
13 SELECT
14     Sex,
15     COUNT(*) as total_people,
16     SUM(Survived) as survivors,
17     ROUND((SUM(Survived) / COUNT(*) * 100, 2) as survival_rate_percent
18 FROM titanic
19 GROUP BY Sex;
20
```

Results (just now)

Table	Chart	2 rows	80ms				
#	SEX	#	TOTAL_PEOPLE	#	SURVIVORS	#	SURVIVAL_RATE_PERCENT
1	female		314		233		74.20
2	male		577		109		18.89

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries.sql
- Titanic_Queries.sql**
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

DEMO_SC

Tables 2

Stages 1

My Workspace > Titanic_Queries.sql

```
22 SELECT
23     Pclass,
24     COUNT(*) as total_passengers,
25     SUM(Survived) as survivors,
26     ROUND((SUM(Survived) / COUNT(*)) * 100, 2) as survival_rate_percent
27 FROM titanic
28 GROUP BY Pclass
29 ORDER BY Pclass;
30
```

Results (2 minutes ago)

Table Chart

3 rows 95ms

#	PCLASS	#	TOTAL_PASSENGERS	#	SURVIVORS	#	SURVIVAL_RATE_PERCENT
1	1		216		136		62.96
2	2		184		87		47.28
3	3		491		119		24.24

Query History

Feedback

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries.sql
- Titanic_Queries.sql**
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

DEMO_SC

Tables 2

My Workspace > Titanic_Queries.sql

```
32 SELECT
33     CASE WHEN Survived = 1 THEN 'Survived' ELSE 'Perished' END as status,
34     ROUND(AVG(Age), 1) as avg_age,
35     MIN(Age) as youngest,
36     MAX(Age) as oldest
37 FROM titanic
38 GROUP BY Survived;
39
40 -- 6. Ticket Prices: Who paid the most?
```

Results (just now)

Table Chart

2 rows 716ms

#	STATUS	#	AVG_AGE	#	YOUNGEST	#	OLDEST
1	Perished		30.6		1		
2	Survived		28.3		0.42		

Query History

Feedback

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries.sql
- Titanic_Queries.sql**
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

DEMO_SC

Tables 2

Stages 1

My Workspace > Titanic_Queries.sql

```
54 LIMIT 10;
55
56 -- 8. Embarkation: Where did most people get on?
57 -- C = Cherbourg, Q = Queenstown, S = Southampton
58 SELECT Embarked, COUNT(*) as count
59 FROM titanic
60 GROUP BY Embarked
61 ORDER BY count DESC;
```

Results (just now)

Table Chart

4 rows 90ms

#	EMBARKED	#	COUNT
1	S		644
2	C		168
3	Q		77
4	null		2

Query History

Feedback

Creating new stage and using COPY for manual import

The screenshot shows the Snowflake SQL Editor interface. The top bar displays the workspace name 'My Workspace' and the active file 'Titanic_manual_load.sql'. The left sidebar contains a 'Database Explorer' panel with a search bar and a list of objects, including 'MY_INTERNAL_STAGE' and 'TITANIC_STAGE'. The main editor area shows a SQL query that creates a stage named 'titanic_stage' with a file format of 'PARQUET'. The query is as follows:

```
1  USE DATABASE demo_db;
2  USE SCHEMA demo_sc;
3
4  -- 1. Create a Specific Stage for this dataset
5  -- A dedicated stage helps keep your Titanic data isolated from other uploads.
6  CREATE OR REPLACE STAGE titanic_stage
7    FILE_FORMAT = (TYPE = 'PARQUET');
8
9  -- IMPORTANT:
```

Below the query, the 'Results (5 minutes ago)' panel shows a single row with the status 'Stage area TITANIC_STAGE successfully created.' The 'Query History' panel at the bottom shows the executed query.

The screenshot shows the 'Upload Your Files' dialog in the Snowflake interface. The dialog is titled 'Upload Your Files' and shows a file named 'titanic.parquet' with a size of 39.1KB. Below the file list, there are options to 'Select database, schema and stage' and 'Specify the path to an existing folder or create a new one (optional)'. The 'Schema' is set to 'DEMO_DB.DEMO_SC' and the 'Stage' is set to 'TITANIC_STAGE'. The 'Path' field is empty. The dialog also includes a 'Close' button and an 'Upload' button.

Creating table

My Workspace

Search files Ctrl+P

+ Add new

Basic_Queries.sql

Create_DB.sql

load_to_azure.sql

Monthwise_Hires.sql

Temporary_Table_Queries.sql

Titanic_manual_Load.sql

Titanic_Queries.sql

Titanic_Table_Creation.sql

Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

MY_INTERNAL_STAGE

TITANIC_STAGE

File Formats 2

INFORMATION_SCHEMA

Home

Create_DB.sql

Monthwise_Hires.sql

Basic_Queries.sql

Verifying_Data_Load.sql

load_to_azure.sql

Tita

ACCOUNTADMIN COMPUTE_WH(X-Small)

DEMO_DB

DEMO_SC

Share

...

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

-- 2. Create the Target Table

CREATE OR REPLACE TABLE titanic_raw (

passenger_id INT,

survived BOOLEAN, -- Loading as Boolean for better Logic later

pclass INT,

full_name STRING,

sex STRING,

age FLOAT,

fare FLOAT

)

-- 3. Load Data using "\$" (Transformation)

Copy data from Source data into stage object (\$\$)

Results (just now)

Table Chart

1 row 148ms

Table

status

1

Table TITANIC_RAW successfully created.

Query History

Feedback

Loading data manually into table

My Workspace

Search files Ctrl+P

+ Add new

Basic_Queries.sql

Create_DB.sql

load_to_azure.sql

Monthwise_Hires.sql

Temporary_Table_Queries.sql

Titanic_manual_Load.sql

Titanic_Queries.sql

Titanic_Table_Creation.sql

Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

MY_INTERNAL_STAGE

TITANIC_STAGE

File Formats 2

INFORMATION_SCHEMA

Basic_Queries.sql

Verifying_Data_Load.sql

load_to_azure.sql

Titanic_Table_Creation.sql

Titanic_Queries.sql

Titanic_manual_Load

ACCOUNTADMIN COMPUTE_WH(X-Small)

DEMO_DB

DEMO_SC

Share

...

24

25

26

27

28

29

30

31

32

33

34

35

36

37

COPY INTO titanic_raw (passenger_id, survived, pclass, full_name, sex, age, fare)

FROM (

SELECT

\$1:PassengerId::INT,

CASE WHEN \$1:Survived::INT = 1 THEN TRUE ELSE FALSE END, -- Transforming logic during load

\$1:Pclass::INT,

\$1:Name::STRING,

\$1:Sex::STRING,

\$1:Age::FLOAT,

\$1:Fare::FLOAT

FROM @titanic_stage/titanic.parquet

)

FILE_FORMAT = (TYPE = 'PARQUET')

ON_ERROR = 'CONTINUE';

Results (just now)

Table Chart

1 row 1.8s

Table

file

status

#

rows_parsed

#

rows_loaded

#

error_limit

#

errors_seen

file

1

titanic_stage/titanic.pa

LOADED

891

891

891

0

null

Query History

Feedback

My Workspace

Basic_Queries.sql Verifying_Data_Load.sql load_to_azure.sql Titanic_Table_Creation.sql Titanic_Queries.sql Titanic_manual_load

Search files Ctrl+P

+ Add new

Basic_Queries.sql
Create_DB.sql
load_to_azure.sql
Monthwise_Hires.sql
Temporary_Table_Queries.sql
Titanic_manual_load.sql
Titanic_Queries.sql
Titanic_Table_Creation.sql
Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

MY_INTERNAL_STAGE
TITANIC_STAGE
File Formats 2
INFORMATION_SCHEMA

My Workspace > Titanic_manual_load.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB DEMO_SC Share

```
34 FROM @titanic_stage/titanic.parquet
35 )
36 FILE_FORMAT = (TYPE = 'PARQUET')
37 ON_ERROR = 'CONTINUE';
38
39 -- 4. Verify the Load
40 SELECT * FROM titanic_raw LIMIT 10;
41
```

Results (just now)

Table Chart 10 rows 773ms

#	PASSENGER_ID	01 SURVIVED	#	PCLASS	A FULL_NAME	A SEX	#	AGE	#	FARE	
1	10	false	50.0%	1	3	Allen, Mr. William H...	10.0%	fe...	50.0%	7.25	71.2...
1	1	FALSE	50.0%	1	3	Braund, Mr. Owen Harris	10.0%	fe...	50.0%	7.25	71.2...
2	2	TRUE	50.0%	1	3	Cummings, Mrs. John Bradley (Fl...	10.0%	fe...	50.0%	7.25	71.2...

Query History

Feedback

Creating a view

My Workspace

Basic_Queries.sql Verifying_Data_Load.sql load_to_azure.sql Titanic_Table_Creation.sql Titanic_Queries.sql Titanic_manual_load

Search files Ctrl+P

+ Add new

Basic_Queries.sql
Create_DB.sql
load_to_azure.sql
Monthwise_Hires.sql
Temporary_Table_Queries.sql
Titanic_manual_load.sql
Titanic_Queries.sql
Titanic_Table_Creation.sql
Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

MY_INTERNAL_STAGE
TITANIC_STAGE
File Formats 2
INFORMATION_SCHEMA

My Workspace > Titanic_manual_load.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB DEMO_SC Share

```
44 CREATE OR REPLACE VIEW v_titanic_summary AS
45 SELECT
46     passenger_id,
47     full_name,
48     sex,
49     age,
50     -- Create a derived column for Age Group
51     CASE
52         WHEN age < 18 THEN 'Child'
53         WHEN age BETWEEN 18 AND 60 THEN 'Adult'
54         ELSE 'Senior'
55     END AS age_category,
56     pclass,
57     fare,
58     survived
59 FROM titanic_raw;
60
```

Results (just now)

Table Chart 1 row 134ms

#	status
1	View V.TITANIC-SUMMARY successfully created.

Query History

Feedback

My Workspace

Basic_Queries.sql Verifying_Data_Load.sql load_to_azure.sql Titanic_Table_Creation.sql Titanic_Queries.sql Titanic_manual_load

Search files Ctrl+P

+ Add new

Basic_Queries.sql
Create_DB.sql
load_to_azure.sql
Monthwise_Hires.sql
Temporary_Table_Queries.sql
Titanic_manual_load.sql
Titanic_Queries.sql
Titanic_Table_Creation.sql
Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

MY_INTERNAL_STAGE
TITANIC_STAGE
File Formats 2
INFORMATION_SCHEMA

My Workspace > Titanic_manual_load.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB DEMO_SC Share

```
58 survived
59 FROM titanic_raw;
60
61 -- 6. Test the View
62 SELECT * FROM v_titanic_summary
63 WHERE age_category = 'Child'
64 ORDER BY fare DESC;
```

Results (just now)

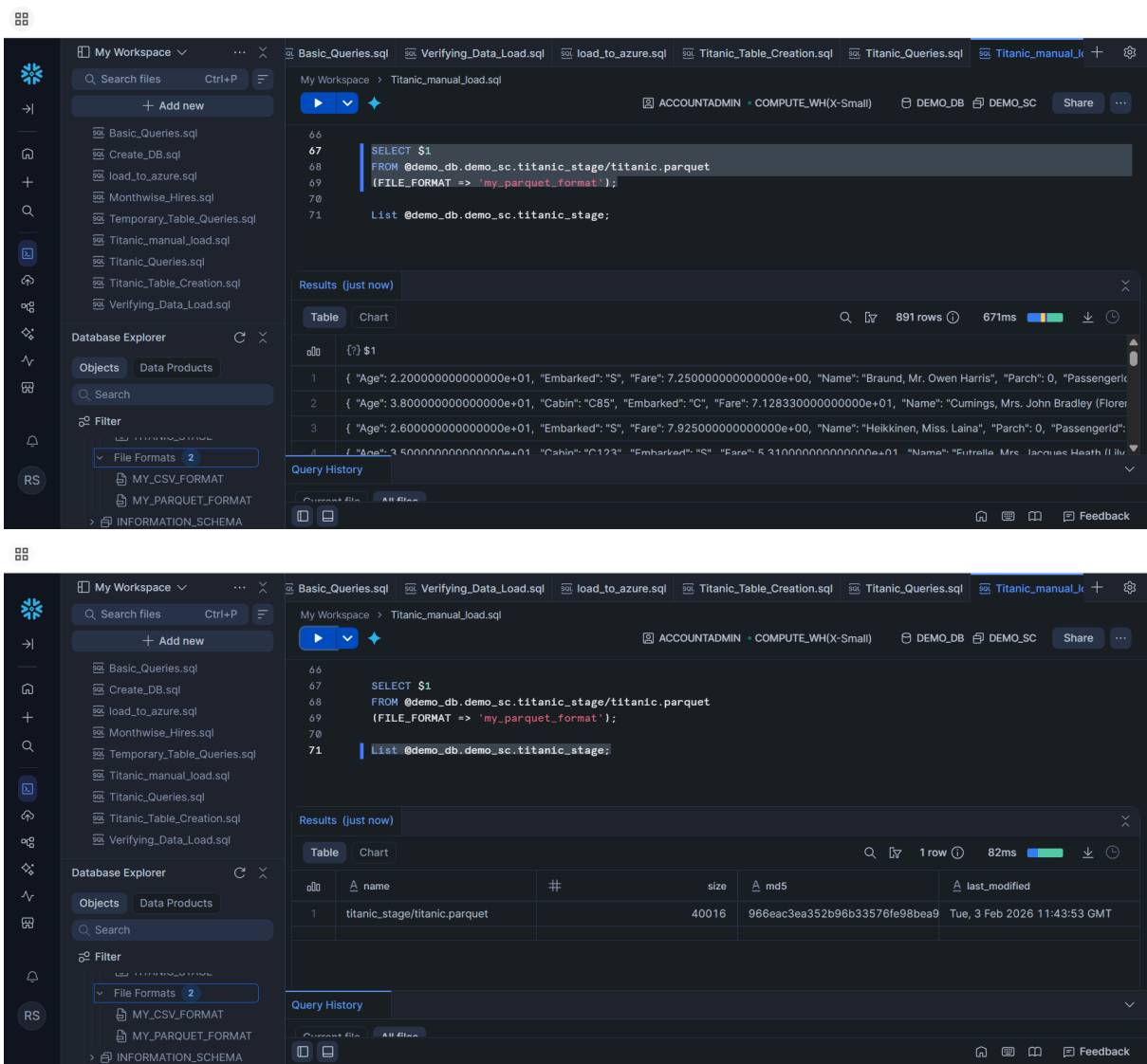
Table Chart 113 rows 143ms

#	PASSENGER_ID	A FULL_NAME	A SEX	#	AGE	A AGE_CATEGORY	#	PCLASS	#	FARE	01 SURVIVED
1	690	Madill, Miss. George Alexandra	female	15	Child	100.0%	1	3	211.3375	TRUE	
2	298	Allison, Miss. Helen Loraine	female	2	Child	100.0%	1	3	151.55	FALSE	
3	306	Allison, Master. Hudson Trevor	male	0.92	Child	100.0%	1	3	151.55	TRUE	

Query History

Feedback

Raw stage created



Summary

This project executes two distinct data engineering pipelines: one for a CSV-based Employee dataset and another for a Parquet-based Titanic dataset.

Important Note on Data Ingestion: While the repository includes scripts for cloud integration (Azure) and CLI-based loading, administrative restrictions prevented the use of these methods.

Data was ingested manually via the Snowflake Web UI.

Employee CSVs: Uploaded directly into the target table (EMP).

Titanic Parquet: Uploaded to a Snowflake Stage first, then processed via SQL.

1. Infrastructure & Setup

Create_DB.sql

Purpose: Initializes the foundational database environment.

Operations: Creates the DEMO_DB database and DEMO_SC schema. It defines the EMP table structure (columns for name, email, address, etc.) to prepare for the CSV data load.

load_to_azure.sql

Purpose: Intended to set up a secure connection to external cloud storage.

Operations: Defines a STORAGE INTEGRATION pointing to an Azure Blob container (azure://ronitshettystorage...).

Status: Not executed due to the administrative restrictions mentioned above.

2. Employee Data Pipeline (CSV)

Data Loading (Web UI)

The 5 separate CSV files (employees01–employees05) were manually uploaded via the Web UI directly into the EMP table defined in Create_DB.sql.

Verifying_Data_Load.sql

Purpose: Quality assurance immediately following the manual upload.

Operations: Checks record counts (COUNT(*)), retrieves a sample of rows (LIMIT 10), and verifies string formatting by concatenating names to ensure columns mapped correctly.

Basic_Queries.sql

Purpose: Fundamental exploration of the employee roster.

Operations: Performs standard filtering (finding employees in specific cities), sorting (alphabetical lists), and aggregation (counting employees per city to identify hubs).

Temporary_Table_Queries.sql

Purpose: Advanced, session-based analytics using temporary storage.

Operations: Creates a TEMPORARY TABLE (emp_analysis_temp) that enriches the raw data with calculated columns: email_domain (parsed via SPLIT_PART) and days_employed (calculated via DATEDIFF).

Executes complex logic on this temporary data, such as categorizing staff as 'Veteran' or 'Newcomer', analyzing seasonal hiring (Q4), and filtering via wildcards (LIKE '%Apt%').

Monthwise_Hires.sql

Purpose: Time-series reporting.

Operations: Aggregates hiring data specifically for the year 2017, grouping by MONTHNAME to visualize recruitment spikes.

3. Titanic Data Pipeline (Parquet)

Titanic_Table_Creation.sql

Purpose: Environment setup specific to binary data.

Operations: Defines a custom file format `my_parquet_format` to handle Parquet compression and creates the target schema for the titanic table.

Titanic_manual_load.sql

Purpose: The core ELT (Extract, Load, Transform) script.

Operations: Creates a named internal stage `titanic_stage`.

Context: The `titanic.parquet` file was uploaded to this stage via the Web UI.

Executes a `COPY INTO` command that transforms the data during the load. It queries the staged file using variant syntax (`$1:Survived`), converts data types (e.g., Integer 1/0 to Boolean True/False), and inserts the cleaned data into `titanic_raw`.

Creates a View `v_titanic_summary` to abstract business logic, such as categorizing passengers into 'Child', 'Adult', or 'Senior' based on age.

Titanic_Queries.sql

Purpose: Deep analytical insights into the passenger manifest.

Operations: Calculates survival rates broken down by Gender and Class, computes average ages of survivors vs. victims, analyzes ticket costs, and determines family sizes by adding sibling/spouse and parent/child counts.

Note on Preliminary Data Stage:

The preliminary data loaded is currently in its raw state (Bronze Layer). It was inspected directly from the stage using the following commands:

```
SELECT $1
FROM @demo_db.demo_sc.titanic_stage/titanic.parquet
(FILE_FORMAT => 'my_parquet_format');
```

```
List @demo_db.demo_sc.titanic_stage;
```

The subsequent Silver (cleaning and standardization) and Gold (aggregation and modeling) stages will be performed later.

04/02/2026

Snowflake Screenshots

Name: Ronit Shetty

Creating zero copy clone and running queries

This screenshot shows the Snowflake IDE interface. On the left, the 'Database Explorer' pane displays a list of files in the 'My Workspace' folder, including 'Basic_Queries.sql', 'Create_DB.sql', 'load_to_azure.sql', 'Monthwise_Hires.sql', 'Temporary_Table_Queries...', 'Titanic_manual_Load.sql', 'Titanic_Queries.sql', 'Titanic_Table_Creation.sql', and 'Verifying_Data_Load.sql'. The 'Database Explorer' pane also shows a search bar and a filter section. The main editor pane displays the 'Zero_Copy_Cloning.sql' file with the following SQL code:

```
1 CREATE OR REPLACE DATABASE demo_db_clone CLONE demo_db;
2 -- Creates an isolated clone of DEMO_DB instantly using zero-copy cloning.
3
4 USE DATABASE demo_db_clone;
5 -- Switches the current session context to the new clone database.
6
7 USE SCHEMA demo_sc;
8 -- Switches the schema context to demo_sc within the clone.
```

The 'Results (just now)' pane shows a single row with the status 'Database DEMO_DB_CLONE successfully created.' The 'Query History' pane is empty.

This screenshot shows the Snowflake IDE interface after the query execution. The main editor pane displays the same SQL code as the previous screenshot, but the 'Results (just now)' pane now shows a single row with the status 'Statement executed successfully.' The 'Query History' pane is also empty.



My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Titanic_manual_load.sql
- Titanic_Queries.sql
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

- TITANIC_STAGE
- File Formats 2
 - MY_CSV_FORMAT
 - MY_PARQUET_FORMAT

Zero_Copy_Cloning.sql

My Workspace > Zero_Copy_Cloning.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB_CLONE DEMO_SC Share ...

```
12
13 SELECT COUNT(*) AS count_before_delete FROM emp;
14 -- Checks the initial row count of the 'emp' table in the clone.
15
16 DELETE FROM emp WHERE city = 'Miami';
17 -- Deletes rows from the clone to simulate a destructive change.
18
19 SELECT COUNT(*) AS count_after_delete FROM emp;
20 -- Verifies the row count has decreased in the clone.
```

Results (just now)

Table Chart

1 row 403ms

#	COUNT_BEFORE_DELETE
1	25

Query History

Current file All files

Feedback



My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Titanic_manual_load.sql
- Titanic_Queries.sql
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

- TITANIC_STAGE
- File Formats 2
 - MY_CSV_FORMAT
 - MY_PARQUET_FORMAT

Zero_Copy_Cloning.sql

My Workspace > Zero_Copy_Cloning.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB_CLONE DEMO_SC Share ...

```
12
13 SELECT COUNT(*) AS count_before_delete FROM emp;
14 -- Checks the initial row count of the 'emp' table in the clone.
15
16 DELETE FROM emp WHERE city = 'Miami';
17 -- Deletes rows from the clone to simulate a destructive change.
18
19 SELECT COUNT(*) AS count_after_delete FROM emp;
20 -- Verifies the row count has decreased in the clone.
```

Results (just now)

Table Chart

1 row 1.9s

#	number of rows deleted
1	1

Query History

Current file All files

Feedback



My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Titanic_manual_load.sql
- Titanic_Queries.sql
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

- TITANIC_STAGE
- File Formats 2
 - MY_CSV_FORMAT
 - MY_PARQUET_FORMAT

Zero_Copy_Cloning.sql

My Workspace > Zero_Copy_Cloning.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB_CLONE DEMO_SC Share ...

```
18
19 SELECT COUNT(*) AS count_after_delete FROM emp;
20 -- Verifies the row count has decreased in the clone.
21
22 SELECT COUNT(*) AS original_db_count FROM demo_db.demo_sc.emp;
23 -- Verifies the original database remains unchanged (should match the initial count).
24
25 SET safe_timestamp = CURRENT_TIMESTAMP();
```

Results (just now)

Table Chart

1 row 80ms

#	COUNT_AFTER_DELETE
1	24

Query History

Current file All files

Feedback



My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Titanic_manual_Load.sql
- Titanic_Queries.sql
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

- TITANIC_STAGE
- File Formats 2
 - MY_CSV_FORMAT
 - MY_PARQUET_FORMAT

Zero_Copy_Cloning.sql

My Workspace > Zero_Copy_Cloning.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB_CLONE DEMO_SC Share ...

```
18
19 SELECT COUNT(*) AS count_after_delete FROM emp;
20 -- Verifies the row count has decreased in the clone.
21
22 SELECT COUNT(*) AS original_db_count FROM demo_db.demo_sc.emp;
23 -- Verifies the original database remains unchanged (should match the initial count).
24
25 SET safe_timestamp = CURRENT_TIMESTAMP();
```

Results (just now)

Table Chart

1 row 69ms

#	ORIGINAL_DB_COUNT
1	25

Query History

Current file All files

Feedback



My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Titanic_manual_Load.sql
- Titanic_Queries.sql
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

- TITANIC_STAGE
- File Formats 2
 - MY_CSV_FORMAT
 - MY_PARQUET_FORMAT

Zero_Copy_Cloning.sql

My Workspace > Zero_Copy_Cloning.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB_CLONE DEMO_SC Share ...

```
24
25 SET safe_timestamp = CURRENT_TIMESTAMP();
26 -- Captures the current timestamp to mark a safe point for time travel.
27
28 DROP TABLE titanic;
29 -- Simulates a disaster by dropping the 'titanic' table in the clone.
30
31 CREATE OR REPLACE TABLE titanic_restored CLONE titanic AT (TIMESTAMP => $safe_timestamp);
32 -- Restores the dropped table using the Point in Time feature from the cloned database.
```

Results (just now)

Table Chart

1 row 85ms

status
Statement executed successfully.

Query History

Current file All files

Feedback



My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Titanic_manual_Load.sql
- Titanic_Queries.sql
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

- TITANIC_STAGE
- File Formats 2
 - MY_CSV_FORMAT
 - MY_PARQUET_FORMAT

Zero_Copy_Cloning.sql

My Workspace > Zero_Copy_Cloning.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB_CLONE DEMO_SC Share ...

```
24
25 SET safe_timestamp = CURRENT_TIMESTAMP();
26 -- Captures the current timestamp to mark a safe point for time travel.
27
28 DROP TABLE titanic;
29 -- Simulates a disaster by dropping the 'titanic' table in the clone.
30
31 CREATE OR REPLACE TABLE titanic_restored CLONE titanic AT (TIMESTAMP => $safe_timestamp);
32 -- Restores the dropped table using the Point in Time feature from the cloned database.
```

Results (just now)

Table Chart

1 row 106ms

status
TITANIC successfully dropped.

Query History

Current file All files

Feedback

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Titanic_manual_load.sql
- Titanic_Queries.sql
- Titanic_Table_Creation.sql
- Verifying_Data_Load.sql

Database Explorer

Objects Data Products

Search

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Zero_Copy_Cloning.sql

My Workspace > Zero_Copy_Cloning.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB_CLONE DEMO_SC Share

```
24
25 SET safe_timestamp = CURRENT_TIMESTAMP();
26 -- Captures the current timestamp to mark a safe point for time travel.
27
28 DROP TABLE titanic;
29 -- Simulates a disaster by dropping the 'titanic' table in the clone.
30
31 UNDROP TABLE titanic;
32
33 CREATE OR REPLACE TABLE titanic_restored CLONE titanic AT (TIMESTAMP => $safe_timestamp);
34 -- Restores the dropped table using Time Travel by cloning from the saved timestamp.
35
```

Results (1 minute ago)

Table Chart

1 row 68ms

#	status
1	Table TITANIC successfully restored.

Query History

Current file All files

Feedback

Time travel

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_load.sql
- Titanic_Queries.sql
- Titanic_Table_Creation.sql

Database Explorer

Objects Data Products

Search

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Zero_Copy_Cloning.sql Time_Travel.sql Titanic_manual_load.sql Titanic_Queries.sql Titanic_Table_Creation.sql

My Workspace > Time_Travel.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB_CLONE DEMO_SC Share

```
1 USE DATABASE demo_db_clone;
2 USE SCHEMA demo_sc;
3
4 -- 1. ANCHOR: Capture the current time BEFORE the mistake
5 SET safe_point = CURRENT_TIMESTAMP();
6 -- We save the exact time right now so we can travel back to it later.
7
8 -- 2. THE MISTAKE: "Accidentally" update all passengers to Survived = 0
9 UPDATE titanic
10 SET Survived = 0;
11
```

Results (just now)

Table Chart

1 row 47ms

#	status
1	Statement executed successfully.

Query History

Current file All files

Feedback

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_load.sql
- Titanic_Queries.sql
- Titanic_Table_Creation.sql

Database Explorer

Objects Data Products

Search

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Zero_Copy_Cloning.sql Time_Travel.sql Titanic_manual_load.sql Titanic_Queries.sql Titanic_Table_Creation.sql

My Workspace > Time_Travel.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB_CLONE DEMO_SC Share

```
6 -- We save the exact time right now so we can travel back to it later.
7
8 -- 2. THE MISTAKE: "Accidentally" update all passengers to Survived = 0
9 UPDATE titanic
10 SET Survived = 0;
11
12 -- 3. VERIFY THE DAMAGE
13 SELECT Survived, COUNT(*)
14 FROM titanic
15 GROUP BY Survived;
16 -- Result: You will see 891 rows with '0' (Deceased). The data is ruined.
17
```

Results (just now)

Table Chart

1 row 1.8s

#	number of rows updated	number of multi-joined rows updated
1	891	0

Query History

Current file All files

Feedback

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql**
- Titanic_manual_Load.sql
- Titanic_Queries.sql
- Titanic_Table_Creation.sql

Database Explorer

Objects Data Products

Search

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Zero_Copy_Cloning.sql Time_Travel.sql Titanic_manual_Load.sql Titanic_Queries.sql Titanic_Table_Creation.sql

My Workspace > Time_Travel.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB_CLONE DEMO_SC Share ...

```
6 -- We save the exact time right now so we can travel back to it later.
7
8 -- 2. THE MISTAKE: "Accidentally" update all passengers to Survived = 0
9 UPDATE titanic
10 SET Survived = 0;
11
12 -- 3. VERIFY THE DAMAGE
13 SELECT Survived, COUNT(*)
14 FROM titanic
15 GROUP BY Survived;
16 -- Result: You will see 891 rows with '0' (Deceased). The data is ruined.
17
```

Results (just now)

Table Chart

1 row 115ms

#	SURVIVED	#	COUNT(*)
1	0		891

Query History

Current file All files

Feedback

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql**
- Titanic_manual_Load.sql
- Titanic_Queries.sql
- Titanic_Table_Creation.sql

Database Explorer

Objects Data Products

Search

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Zero_Copy_Cloning.sql Time_Travel.sql Titanic_manual_Load.sql Titanic_Queries.sql Titanic_Table_Creation.sql

My Workspace > Time_Travel.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB_CLONE DEMO_SC Share ...

```
14 FROM titanic
15 GROUP BY Survived;
16 -- Result: You will see 891 rows with '0' (Deceased). The data is ruined.
17
18 -- 4. TIME TRAVEL: Query the table as it was at our 'safe_point'
19 SELECT Survived, COUNT(*)
20 FROM titanic AT(TIMESTAMP => $safe_point)
21 GROUP BY Survived;
22 -- Result: You will see the original split (e.g., 342 Survived, 549 Deceased).
23
24 -- 5. THE FIX: Restore the table using the Time Travel data
```

Results (just now)

Table Chart

2 rows 72ms

#	SURVIVED	#	COUNT(*)
1	1		342
2	0		549

Query History

Current file All files

Feedback

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql**
- Titanic_manual_Load.sql
- Titanic_Queries.sql
- Titanic_Table_Creation.sql

Database Explorer

Objects Data Products

Search

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Zero_Copy_Cloning.sql Time_Travel.sql Titanic_manual_Load.sql Titanic_Queries.sql Titanic_Table_Creation.sql

My Workspace > Time_Travel.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB_CLONE DEMO_SC Share ...

```
22 -- Result: You will see the original split (e.g., 342 Survived, 549 Deceased).
23
24 -- 5. THE FIX: Restore the table using the Time Travel data
25 CREATE OR REPLACE TABLE titanic
26 AS
27 SELECT * FROM titanic AT(TIMESTAMP => $safe_point);
28 -- We completely overwrite the broken table with the data from the past.
```

Results (just now)

Table Chart

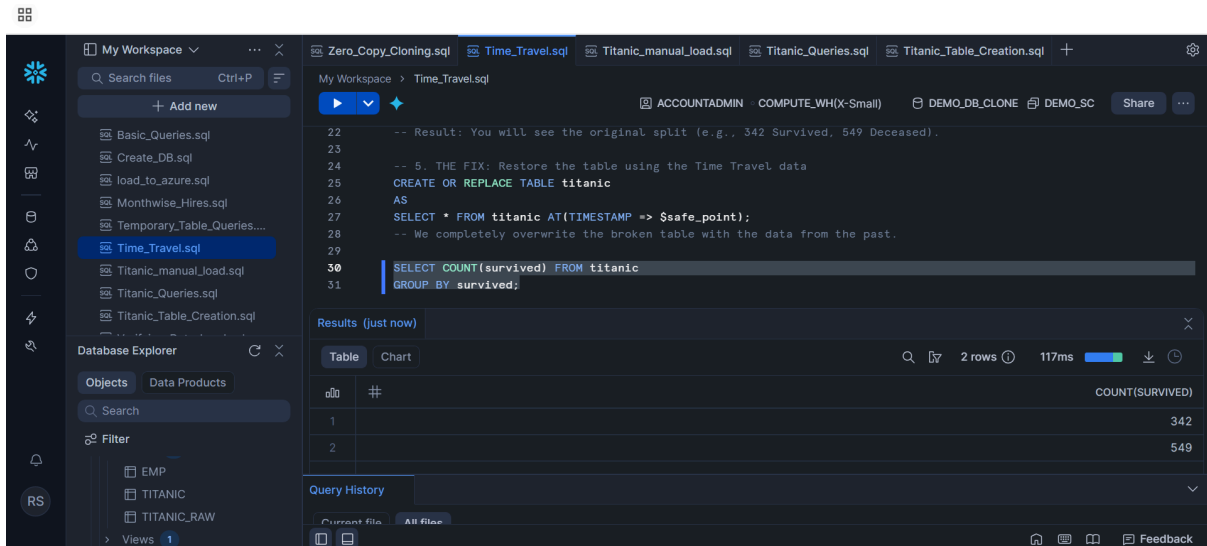
1 row 1.5s

#	status
1	Table TITANIC successfully created.

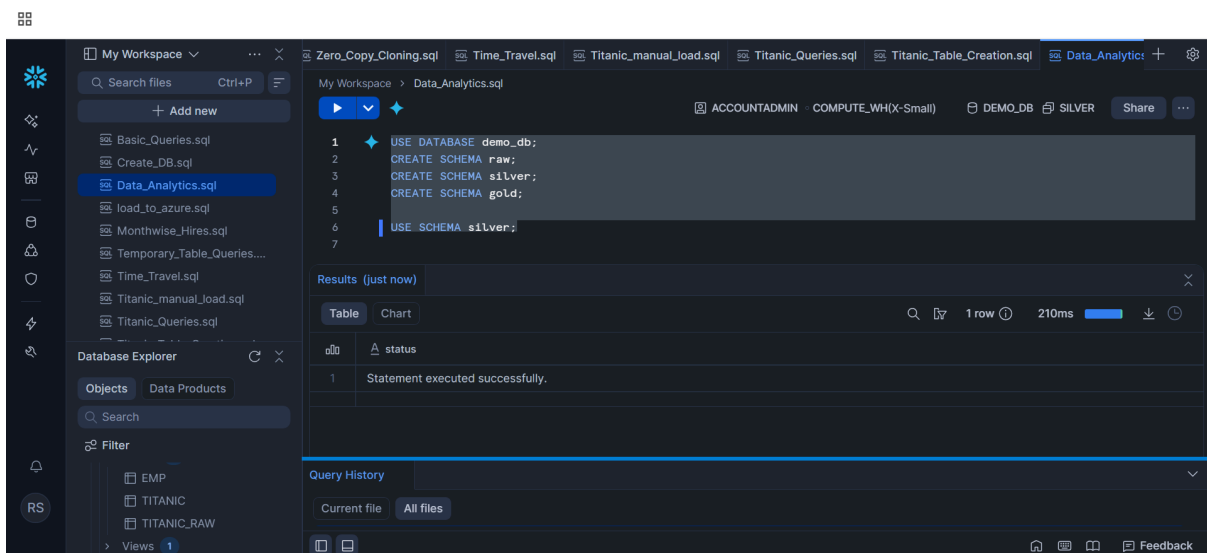
Query History

Current file All files

Feedback



Creating stages for data analysis



Connection to azure established

The consent link has to be accessed by the owner of the resources, and then the person can be added as a blob storage contributor or any other role with the help of their azure multi tenant app name.

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- Data_Analytics.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_Load.sql
- Titanic_Queries.sql

Database Explorer

Objects Data Products

Search

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Data_Analytics.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB SILVER Share

```
7
8 create or replace storage integration azure_int
9 type = external_stage
10 storage_provider = azure
11 enabled = true
12 azure_tenant_id = '42d74fda-bcb1-41e8-8d81-049031eb498a'
13 storage_allowed_locations =
14 ('azure://snowflakebucket.blob.core.windows.net/employee','azure://snowflakebucket.blob.core.windows.net/parquetdata');
15
16 desc integration azure_int;
```

Results (just now)

Table Chart 9 rows 30ms

	property	property_type	property_value	property_default
1	ENABLED	Boolean	true	false
2	STORAGE_PROVIDER	String	AZURE	
3	STORAGE_ALLOWED_LOCATIONS	List	azure://snowflakebucket.blob.core.wi	

Count 2

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- Data_Analytics.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_Load.sql
- Titanic_Queries.sql

Database Explorer

Objects Data Products

Search

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Data_Analytics.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB SILVER Share

```
16
17 CREATE FILE FORMAT "DEMO_DB"."PUBLIC".MY_PARQUET_STAGE TYPE = 'PARQUET' COMPRESSION = 'AUTO' BINARY_AS_TEXT =
18 TRUE;
19 create or replace stage demo_db.public.my_azure_parquet_stage_titanic storage_integration = azure_int url =
20 'azure://snowflakebucket.blob.core.windows.net/parquetdata/titanic.parquet';
21
22 CREATE OR REPLACE transient table demo_db.public.userdata_parquet_titanic (raw_data_variant)
23
```

Results (just now)

Table Chart 1 row 111ms

	status
1	Stage area MY_AZURE_PARQUET_STAGE_TITANIC successfully created.

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- Data_Analytics.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_Load.sql
- Titanic_Queries.sql

Database Explorer

Objects Data Products

Search

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Data_Analytics.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB SILVER Share

```
16
17 CREATE FILE FORMAT "DEMO_DB"."PUBLIC".MY_PARQUET_STAGE TYPE = 'PARQUET' COMPRESSION = 'AUTO' BINARY_AS_TEXT =
18 TRUE;
19 create or replace stage demo_db.public.my_azure_parquet_stage_titanic storage_integration = azure_int url =
20 'azure://snowflakebucket.blob.core.windows.net/parquetdata/titanic.parquet';
21
22 CREATE OR REPLACE transient table demo_db.public.userdata_parquet_titanic(raw_data variant);
23
24 copy into demo_db.public.userdata_parquet_titanic from @demo_db.public.my_azure_parquet_stage_titanic file_format
25 = demo_db.public.my_parquet_stage;
```

Results (just now)

Table Chart 1 row 125ms

	status
1	Table USERDATA_PARQUET_TITANIC successfully created.

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- Data_Analytics.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_load.sql
- Titanic_Queries.sql

Database Explorer

Objects Data Products

Search

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Data_Analytics.sql

Titanic_manual_load.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB SILVER Share ...

```
'azure://snowflakebucket.blob.core.windows.net/parquetdata/titanic.parquet';

CREATE or REPLACE transient table demo_db.public.userdata_parquet_titanic(raw_data variant);

copy into demo_db.public.userdata_parquet_titanic from @demo_db.public.my_azure_parquet_stage_titanic file_format
= demo_db.public.my_parquet_stage;
```

Results (just now)

Table Chart

1 row 5.9s

#	file	status	#	rows_parsed	#	rows_loaded	#	error_limit	#	errors_seen	#	first
1	azure://snowflakebucket	LOADED		891		891		1		0		null

Query History

Feedback

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- Data_Analytics.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_load.sql
- Titanic_Queries.sql

Database Explorer

Objects Data Products

Search

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Data_Analytics.sql

Titanic_manual_load.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB SILVER Share ...

```
CREATE or REPLACE transient table demo_db.public.userdata_parquet_titanic(raw_data variant);

copy into demo_db.public.userdata_parquet_titanic from @demo_db.public.my_azure_parquet_stage_titanic file_format
= demo_db.public.my_parquet_stage;

SELECT * FROM userdata_parquet_titanic;
```

Results (1 minute ago)

Table Chart

891 rows 235ms

#	RAW_DATA
1	{ "Age": 2.2000000000000000e+01, "Embarked": "S", "Fare": 7.2500000000000000e+00, "Name": "Braund, Mr. Owen Harris", "Parch": 0, "PassengerId": 1 }
2	{ "Age": 3.8000000000000000e+01, "Cabin": "C85", "Embarked": "C", "Fare": 7.1283300000000000e+01, "Name": "Cumings, Mrs. John Bradley (Floer)", "Parch": 0, "PassengerId": 2 }
3	{ "Age": 2.6000000000000000e+01, "Embarked": "S", "Fare": 7.9250000000000000e+00, "Name": "Heikinen, Miss. Laina", "Parch": 0, "PassengerId": 3 }
4	{ "Age": 3.5000000000000000e+01, "Cabin": "C123", "Embarked": "S", "Fare": 5.3100000000000000e+01, "Name": "Futrelle, Mrs. Jacques Heath (Lily)", "Parch": 0, "PassengerId": 4 }
5	{ "Age": 3.5000000000000000e+01, "Embarked": "S", "Fare": 8.0500000000000000e+00, "Name": "Allen, Mr. William Henry", "Parch": 0, "PassengerId": 5 }

Query History

Feedback

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- Data_Analytics.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_load.sql
- Titanic_Queries.sql

Database Explorer

Objects Data Products

Search

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Data_Analytics.sql

Titanic_manual_load.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB SILVER Share ...

```
create or replace table demo_db.public.user_parsed_data_titanic as
select
$1:Age Age,
$1:Embarked Embarked,
$1:Fare Fare,
$1:Name Name,
$1:Parch Parch,
$1:PassengerId PassengerId,
$1:Pclass Pclass,
$1:Sex Sex,
$1:SibSp SibSp,
$1:Survived Survived
from demo_db.public.userdata_parquet_titanic;
```

Results (just now)

Table Chart

1 row 883ms

#	status
1	Table USER_PARSED_DATA_TITANIC successfully created.

Query History

Feedback

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- Data_Analytics.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_Load.sql
- Titanic_Queries.sql

Database Explorer

Objects Data Products

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Data_Analytics.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB SILVER Share ...

```
39 from demo_db.public.userdata_parquet_titanic;
40
41 CREATE OR REPLACE TABLE demo_db.silver.titanic_passengers_clean (
42   passenger_id INTEGER,
43   passenger_name STRING,
44   sex STRING,
45   age NUMBER(4, 1),
46   passenger_class INTEGER,
47   sibsp INTEGER,
48   parch INTEGER,
49   fare NUMBER(10, 2),
50   embarked STRING,
51   survived BOOLEAN,
52   load_ts TIMESTAMP DEFAULT CURRENT_TIMESTAMP() );
```

Results (just now)

Table Chart

1 row 220ms

Table

1 Table TITANIC.PASSENGERS_CLEAN successfully created.

Query History

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- Data_Analytics.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_Load.sql
- Titanic_Queries.sql

Database Explorer

Objects Data Products

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Data_Analytics.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB SILVER Share ...

```
53
54 INSERT INTO DEMO_DB.SILVER.TITANIC.PASSENGERS_CLEAN SELECT
55   PASSENGERID::INTEGER AS PASSENGER_ID,
56   NAME::STRING AS PASSENGER_NAME,
57   SEX::STRING AS SEX,
58   AGE::NUMBER(4,1) AS AGE,
59   PCLASS::INTEGER AS PASSENGER_CLASS,
60   SIBSP::INTEGER AS SIBSP,
61   PARCH::INTEGER AS PARCH,
62   FARE::NUMBER(10,2) AS FARE,
63   EMBARKED::STRING AS EMBARKED,
64   SURVIVED::INTEGER = 1 AS SURVIVED,
65   CURRENT_TIMESTAMP()
66 FROM
67   DEMO_DB.PUBLIC.USER_PARSED_DATA_TITANIC
68 WHERE PASSENGERID IS NOT NULL;
```

Results (just now)

Table Chart

1 row 1.1s

Table

1 # number of rows inserted 891

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Create_DB.sql
- Data_Analytics.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_Load.sql
- Titanic_Queries.sql

Database Explorer

Objects Data Products

Filter

- EMP
- TITANIC
- TITANIC_RAW

Views 1

Data_Analytics.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB SILVER Share ...

```
62 FARE::NUMBER(10,2) AS FARE,
63 EMBARKED::STRING AS EMBARKED,
64 SURVIVED::INTEGER = 1 AS SURVIVED,
65 CURRENT_TIMESTAMP()
66 FROM
67   DEMO_DB.PUBLIC.USER_PARSED_DATA_TITANIC
68 WHERE PASSENGERID IS NOT NULL;
69
70 SELECT * FROM DEMO_DB.SILVER.TITANIC.PASSENGERS_CLEAN limit 10;
71
```

Results (just now)

Table Chart

10 rows 702ms

Table

#	PASSENGER_ID	PASSENGER_NAME	SEX	#	AGE	#	PASSENGER_CLASS	#	SIBSP	#	PARCH	#	FARE	
1	1	Allen, Mr. William Harris	male	22.0	3	1	0	7.25	S					
2	2	Braund, Mr. Owen Harris	male	22.0	3	1	0	7.25	S					
3	3	Cumings, Mrs. John Bradley	female	38.0	1	1	0	71.28	C					

Removing duplicate values using QUALITY, PARTITION

88

The screenshot shows the Databricks workspace interface. The left sidebar contains a file explorer with various SQL files and a database explorer showing the 'DEMO_DB' schema. The main editor displays a SQL query in 'Data_Analytics.sql' that creates a deduplicated table from the 'TITANIC.PASSENGERS_CLEAN' table. The query uses a window function to identify duplicates based on 'passenger_id' and 'load_ts'. The 'Results' pane shows a single message: 'Table TITANIC.PASSENGERS_DEDUP successfully created.' The status bar indicates the user is 'ACCOUNTADMIN' and the cluster is 'COMPUTE_WH(X-Small)'.

```
69 SELECT * FROM DEMO_DB.SILVER.TITANIC.PASSENGERS_CLEAN limit 10;
70
71
72 CREATE OR REPLACE TABLE demo_db.silver.titanic_passengers_dedup AS
73 SELECT * FROM DEMO_DB.silver.titanic_passengers_clean QUALIFY ROW_NUMBER() OVER (
74 PARTITION BY passenger_id
75 ORDER BY load_ts DESC
76 ) = 1;
77
78
```

Results (just now)

Table	Chart
status	
1	Table TITANIC.PASSENGERS_DEDUP successfully created.

Data Analysis (Gold stage)

88

The screenshot shows the Databricks workspace interface. The left sidebar is similar to the previous one, but the file explorer highlights 'Data_Analysis.sql'. The main editor displays a SQL query in 'Data_Analysis.sql' that calculates the survival rate by age group from the deduplicated passenger data. The query uses a window function to calculate the survival rate. The 'Results' pane shows a single message: 'Table SURVIVAL_BY_AGE_GROUP successfully created.' The status bar indicates the user is 'ACCOUNTADMIN' and the cluster is 'COMPUTE_WH(X-Small)'.

```
22 WHEN survived THEN 1
23 ELSE 0
24 END
25 ) * 100.0/COUNT(*),2
26 ) AS survival_rate_pct
27 FROM demo_db.silver.titanic_passengers_dedup
28 WHERE age IS NOT NULL
29 GROUP BY
30 age_group;
31
```

Results (just now)

Table	Chart
status	
1	Table SURVIVAL_BY_AGE_GROUP successfully created.

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Connecting_Azure_Data_S...
- Create_DB.sql
- Data_Analysis.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

- DEMO_SC
- Tables 3
- EMP

Connecting_Azure_Data_Stages.sql Titanic_manual_load.sql Create_DB.sql Data_Analysis.sql

My Workspace > Data_Analysis.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB GOLD Share ...

```
29 GROUP BY
30 age_group;
31
32 SELECT * FROM DEMO_DB.GOLD.SURVIVAL_BY_AGE_GROUP;
33
```

Results (just now)

Table Chart

4 rows 154ms

	AGE_GROUP	#	TOTAL_PASSENGERS	#	SURVIVORS	#	SURVIVAL_RATE_PCT
1	Teen		71		31		43.66
2	Adult		553		215		38.88
3	Senior		22		5		22.73
4	Child		68		39		57.35

Feedback

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Connecting_Azure_Data_S...
- Create_DB.sql
- Data_Analysis.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

- DEMO_SC
- Tables 3
- EMP

Connecting_Azure_Data_Stages.sql Titanic_manual_load.sql Create_DB.sql Data_Analysis.sql

My Workspace > Data_Analysis.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB GOLD Share ...

```
33
34 CREATE OR REPLACE TABLE DEMO_DB.GOLD.SURVIVAL_BY_CLASS_GENDER AS
35 SELECT passenger_class, sex, COUNT(*) AS total_passengers,
36 SUM( CASE WHEN survived THEN 1 ELSE 0 END) AS survivors,
37 ROUND( SUM( CASE WHEN survived THEN 1 ELSE 0 END) * 100.0/ COUNT(*), 2) AS survival_rate_pct
38 FROM DEMO_DB.SILVER.TITANIC_PASSENGERS_DEDUP GROUP BY passenger_class, sex;
```

Results (just now)

Table Chart

1 row 1.4s

	status
1	Table SURVIVAL_BY_CLASS_GENDER successfully created.

Feedback

My Workspace

Search files Ctrl+P

+ Add new

- Basic_Queries.sql
- Connecting_Azure_Data_S...
- Create_DB.sql
- Data_Analysis.sql
- load_to_azure.sql
- Monthwise_Hires.sql
- Temporary_Table_Queries....
- Time_Travel.sql
- Titanic_manual_load.sql

Database Explorer

Objects Data Products

Search

Filter

DEMO_DB

- DEMO_SC
- Tables 3
- EMP

Connecting_Azure_Data_Stages.sql Titanic_manual_load.sql Create_DB.sql Data_Analysis.sql

My Workspace > Data_Analysis.sql

ACCOUNTADMIN COMPUTE_WH(X-Small) DEMO_DB GOLD Share ...

```
33
34 CREATE OR REPLACE TABLE DEMO_DB.GOLD.SURVIVAL_BY_CLASS_GENDER AS
35 SELECT passenger_class, sex, COUNT(*) AS total_passengers,
36 SUM( CASE WHEN survived THEN 1 ELSE 0 END) AS survivors,
37 ROUND( SUM( CASE WHEN survived THEN 1 ELSE 0 END) * 100.0/ COUNT(*), 2) AS survival_rate_pct
38 FROM DEMO_DB.SILVER.TITANIC_PASSENGERS_DEDUP GROUP BY passenger_class, sex;
39
40 SELECT * FROM DEMO_DB.GOLD.SURVIVAL_BY_CLASS_GENDER;
```

Results (just now)

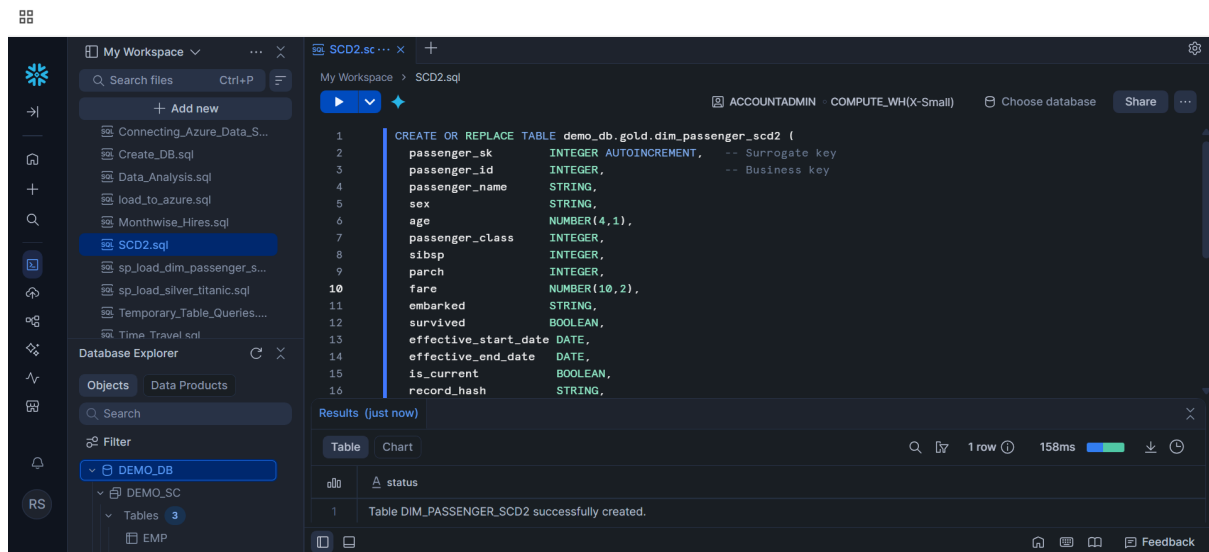
Table Chart

6 rows 96ms

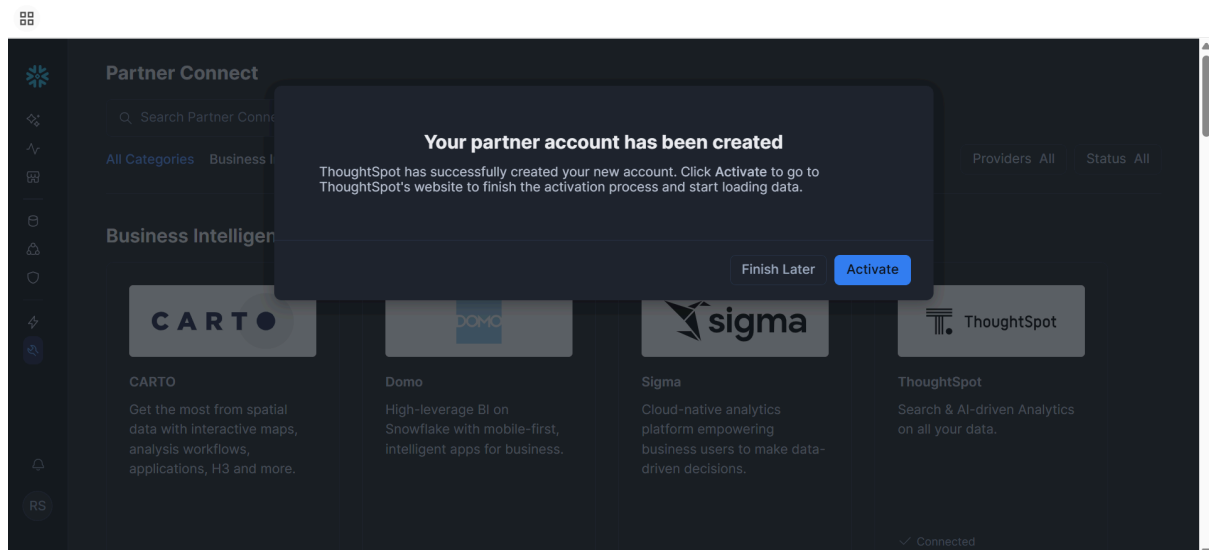
	#	PASSENGER_CLASS	:	SEX	#	TOTAL_PASSENGERS	#	SURVIVORS	#	SURVIVAL_RATE_PCT
1		3		male		347		47		13.54
2		1		female		94		91		96.81
3		1		male		122		45		36.89
4		3		female		144		72		50.00
5		2		female		76		70		92.11

Feedback

Creating scd2 table to store historic data columns



Thoughtspot connection



Only 30 days left in your trial.

Search your library

Invite your team

View pricing

Book expert 1:1 help

TS: AI and BI Stats

Ask a business question in natural language

Home

Spotter

Search data

LIBRARY

Liveboards

Answers

ANALYSIS & ALERTS

Liveboard schedules

Watchlist

Sales By Month

Feb 2026

1.78M

↓ 87.71% vs Jan 2026 (14.46M)

Quantity Purchased By Month

Feb 2026

40.54K

↓ 87.43% vs Jan 2026 (322.61K)

Sales Last Year

Week of 12/29

1.42M

Sample Retail - Sales Performance

Ronit Shetty

ronit.shetty@ext.saint-gobain.com

Profile

Invite teammates

Admin settings

Sign out

Hi Ronit,

Our team of Product Specialists is here to help if you have any questions. Please feel free to send us a chat!

Product Specialists • 1h

Library

All

Yours

All Liveboards

All Answers



Edit connection
Select tables

Snowflake

All

Selected 0

Search database, schema, table

DEMO_DB

DEMO_SC

EMP

TITANIC

TITANIC_RAW

V. TITANIC. SUMMARY

EMP

Search columns...

Column name (0/6 selected)	Data type	Sample data
☐ FIRST_NAME	VARCHAR	Althea
☐ LAST_NAME	VARCHAR	Featherstone
☐ EMAIL	VARCHAR	afeatherstona@sf_tuts.com
☐ STREETADDRESS	VARCHAR	8172 Browning Street, Apt B
☐ CITY	VARCHAR	Colabou

Cancel

Back

Validate and save

Only 30 days left in your trial.

Search your library

Invite your team

View pricing

Book expert 1:1 help

Visualization Dashboard

1. Survival Analysis by Gender

count PASSENGER_ID SURVIVED SEX

750

500

250

0

314

577

female

male

SEX

Pin Save Download Edit Add to Coaching

Ask me a question.

DIM_PASSENGER_SCD2

Spotter responses should be reviewed. Learn more

2. Passenger Distribution by Class

count PASSENGER_IDPASSENGER_CLASS

↶

▼

📊

📈

↗

Total Count PASSENGER_ID	Total PASSENGER_CLASS	
891	2.06K	
AVERAGE 891	TOTAL 2.06K	

Showing 1 of 1 row

📌

Pin

💾

Save

⬇

Download

✎

Edit

+

Add to Coaching

3. Average Age Analysis

Average age by passenger class

average AGEPASSENGER_CLASS

↶

▼

📊

📈

↗

Average AGE	Total PASSENGER_CLASS	
29.7	2.06K	
TABLE AGGREGA... 29.7	TOTAL 2.06K	

Showing 1 of 1 row

📌

Pin

💾

Save

⬇

Download

✎

Edit

4. Average Fare Analysis

Average fare by passenger class

average FAREPASSENGER_CLASS

↶

▼

📊

📈

↷

Average FARE	Total PASSENGER_CLASS	
32.2	2.06K	
TABLE AGGREGA...	TOTAL	
32.2	2.06K	

Showing 1 of 1 row

📌 Pin

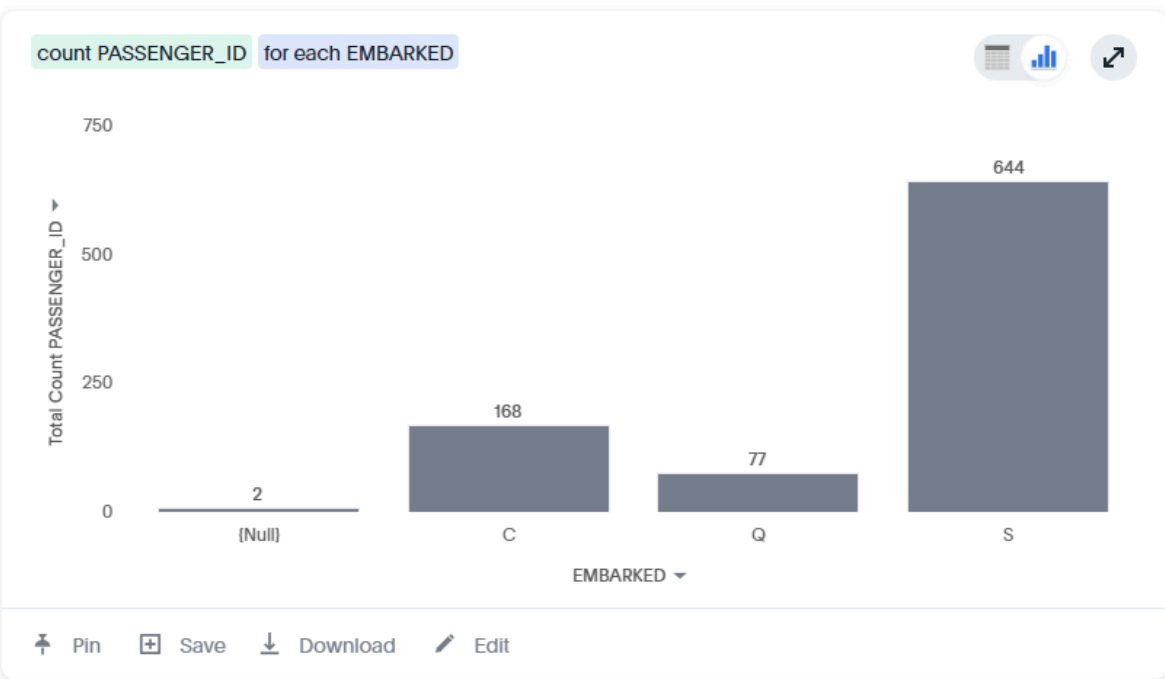
📁 Save

📄 Download

✎ Edit

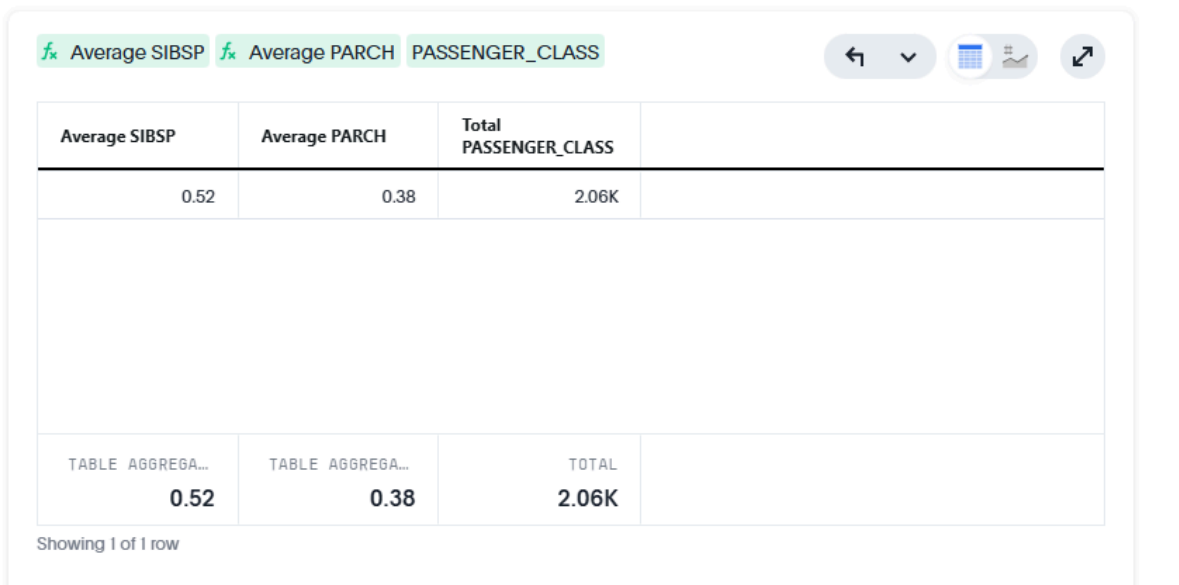
5. Embarkation Port Distribution

Count of passengers by embarkation port



6. Family Composition Patterns

Average number of siblings or spouses and parents or children by passenger class



The screenshot shows a Snowflake SQL Editor interface. At the top, a query is entered: `fx Average SIBSP fx Average PARCH PASSENGER_CLASS`. Below the query, a table displays the results. The table has four columns: 'Average SIBSP', 'Average PARCH', 'Total PASSENGER_CLASS', and an empty column. The first row of data shows values 0.52, 0.38, and 2.06K. Below this, there is a large empty space. At the bottom of the table, a summary row shows 'TABLE AGGREGA...' with values 0.52, 0.38, and 2.06K. The status bar at the bottom indicates 'Showing 1 of 1 row'.

Average SIBSP	Average PARCH	Total PASSENGER_CLASS	
0.52	0.38	2.06K	
TABLE AGGREGA...	TABLE AGGREGA...	TOTAL	
0.52	0.38	2.06K	

Showing 1 of 1 row

Summary of work

1. Infrastructure & Cloud Integration

Azure Connection: Successfully bypassed previous restrictions to configure a STORAGE INTEGRATION (azure_int) connecting Snowflake directly to Azure Blob Storage (azure://snowflakebucket...).

External Stage: Created my_azure_parquet_stage_titanic to ingest raw Parquet files directly from the cloud without manual UI uploads.

Schema Organization: Structured the database into three distinct layers: RAW (Public/Bronze), SILVER (Cleaned), and GOLD (Aggregated/Modeled).

2. Medallion Architecture Implementation

Bronze Layer (Ingestion)

Raw Ingestion: Loaded binary Parquet data into a TRANSIENT variant table (userdata_parquet_titanic) using a custom file format (MY_PARQUET_STAGE).

Parsing: Flattened the semi-structured variant data (\$1:Age, etc.) into a structured tabular format (user_parsed_data_titanic).

Silver Layer (Cleaning & Deduplication)

Data Cleaning: Created titanic_passengers_clean with proper data typing (casting Integers, Booleans, and Strings).

Deduplication Logic: Implemented a robust deduplication strategy using QUALIFY ROW_NUMBER() to retain only the latest record per passenger_id based on the load timestamp.

Automation: Encapsulated this entire logic into a reusable Stored Procedure:
sp_load_silver_titanic().

Gold Layer (Dimensional Modeling & Aggregation)

SCD Type 2 Dimension: Designed dim_passenger_scd2 to track historical changes to passenger data over time. This table includes:

Surrogate Keys: passenger_sk (Auto-increment).

Validity Dates: effective_start_date and effective_end_date.

Current Flag: is_current boolean.

Hash Detection: Used HASH() to compare incoming records against existing ones to detect changes efficiently.

Analytical Marts: Created aggregated tables for reporting:

survival_by_age_group: Segments passengers into Child, Teen, Adult, and Senior categories.

survival_by_class_gender: Calculates survival rates broken down by Pclass and Sex.

Automation: The Gold layer loading, including the complex SCD2 merge logic, is automated via sp_load_dim_passenger_scd2().

3. Advanced Features & Visualization

Zero Copy Cloning: Demonstrated safe environment management by creating demo_db_clone for destructive testing without impacting the main pipeline.

Time Travel & Recovery: Validated Snowflake's fail-safe features by "accidentally" dropping tables/data and restoring them using AT(TIMESTAMP => ...) and UNDROP.

ThoughtSpot Integration: Connected the Gold Layer tables (dim_passenger_scd2, survival_by_*) to ThoughtSpot, enabling self-service BI and visualization on the processed data.

4. Execution Flow

The entire pipeline can now be triggered by running two command calls:

CALL demo_db.silver.sp_load_silver_titanic(); -- Refreshes Silver

CALL demo_db.gold.sp_load_dim_passenger_scd2();